

Copying before selves

What a soup of random machine code tells us about the origin of individuals. A plain-language account of the Algcocell research: where it started, what we did, what we found, and why it matters. Written on 10 October 2026, the morning after the newest result. Technical words are explained where they first appear, and again in the glossary at the end.

Contents

Prologue: a two-byte word that copies itself	2
How to read this	2
Part I. The question	3
1. Which came first, heredity or the individual?	3
2. Why we can't just look	4
Part II. What was known: artificial life	4
3. Life in a computer	4
Part III. Our world	5
4. A primordial soup of Z80 machine code	5
5. How we know something is alive: the culture test	6
Part IV. How we got here	6
6. A bug, a lost observation, and an atlas	6
Part V. What we found	8
7. The first replicators are not individuals	8
8. Then the loop closes	9
9. The rules of the world decide how life begins	10
10. Inside the individual: one byte decides between self-repair and evolution	11
11. The environment chooses, through the company you keep	12
12. A beautiful idea that turned out to be wrong	13
13. The newest finding: how the first individual is born	13
Part VI. Why this matters	15
14. For how we think about the origin of life	15
15. For evolution in general	16
16. For artificial life	17
17. Predictions for chemists	17
Part VII. Honesty and limits	17
18. What we are careful not to claim	17
19. How the work was done	18
20. Where this goes next	18
Glossary	19

Prologue: a two-byte word that copies itself

Inside our simulated worlds lives a creature whose whole recipe is two bytes long. Written in the machine language of a 1976 microprocessor, it reads `01 c5`, and it means: *put the next two bytes into a storage slot; then write that slot out*. The two bytes it picks up are its own. The creature is that two-byte recipe repeated end to end along a short strip of memory. Put it in a crowd of random programs that take turns running each other, and it writes copies of itself into whatever program it happens to be touching, two bytes at a time, until the neighbour has become a copy.

Nobody wrote it. In every one of 170 worlds we examined closely, each starting from pure random noise, with no goal and no reward, this same little program (or a close variant) was the first self-copier to appear, within a few hundred rounds, and it took over. Then, in world after world, something else happened. Its descendants were replaced by programs that keep to themselves.

This document tells the story of how we found that, what it means, and the newest finding: how the first self-contained copier in each world was actually born. It is an exercise in not losing the big picture, so it starts with the big picture.

How to read this

- **Bytes in hexadecimal.** Computers store everything as **bytes**, numbers from 0 to 255. Programmers write each byte as two characters, using the digits 0–9 and then the letters a–f for ten to fifteen (“hexadecimal”, or base 16). So `01 c5` is just the two numbers 1 and 197. Treat these codes as labels, like catalogue numbers. Where we write `XX`, it means “any byte”.
- **A replicator** is anything that makes copies of itself that can in turn make copies.
- **Median** is the middle value: half the worlds were above it and half below. A range of medians such as “0.66–0.73” means the median varied from 0.66 to 0.73 across the different settings we tried (for example, different tape lengths). A share such as 0.66 means 66%.
- **Bits** measure information. One bit is the answer to one yes-or-no question; eight bits pick one item out of 256.
- **A key to the main characters.** We sort replicators into two kinds. **Open** replicators (we also call them **pushers**) let their activity spill into their neighbour. **Closed** replicators, also called **self-confined**, keep their activity inside themselves; we call a closed replicator a **closer** (one that closes, not “one that is nearer”). Closed replicators are our measurable stand-in for **individuals**.
- **Harmless and lethal tar.** Our worlds fill up with debris, mostly zero bytes, which we call tar. On the real chip a zero byte does nothing, so the tar is harmless. In some experiments we change the rules so that running into a zero stops a program dead: lethal tar. Comparing the two turns out to matter a great deal.

Part I. The question

1. Which came first, heredity or the individual?

Every living thing you have ever seen is an **individual**: a cell, or a body made of cells, with an inside and an outside. Inside the boundary are the instructions (the genes) and the machinery that reads and copies them. When a bacterium divides, it copies its own DNA with its own enzymes inside its own membrane and hands a copy to each daughter. Copying and selfhood come as a package.

That package is so universal that it is easy to assume it was there from the start. But it hides a chicken-and-egg question that origin-of-life researchers have argued about for decades:

Which came first: heredity, or the individual?

- **Heredity** means that offspring resemble their parents because information is copied from one to the other. Without heredity there is no evolution: a lucky improvement would vanish instead of being passed on.
- An **individual**, in the sense we use here, is a unit that does its own copying and whose future depends on itself, not on its surroundings. It carries its instructions *and* the machinery that copies them, and what it produces does not depend on which neighbour it happens to meet.

Theorists have given this second idea several names. The biologists Humberto Maturana and Francisco Varela called it *operational closure*; the theoretical biologist Robert Rosen spoke of *closure to efficient causation*; David Krakauer and colleagues turned it into a number with their *information theory of individuality*, in which an individual is something whose future is better predicted by its own past than by its environment. All of these mean, roughly, that what makes the system work is made or decided by the system itself. An individual is a loop that closes on itself.

One view of the origin of life imagines that the first evolving things were individuals of this kind from the start: for example, a membrane bag (a **protocell**) holding a molecule that could copy itself, with its own family line. Another view, associated with the microbiologist **Carl Woese**, holds that the earliest cells were loosely organised and swapped genetic material so freely (biologists call this **horizontal transfer**: genes passing sideways between organisms rather than from parent to child) that there were no stable family lines. Evolution then acted on the community more than on any one cell. Only later, Woese argued, did life cross a “**Darwinian threshold**” into the familiar world of individuals with their own family trees. The philosopher Peter Godfrey-Smith has a name for things that reproduce only with outside help: **scaffolded reproducers**. A virus is the classic modern example: it carries heredity but uses a cell’s machinery to copy itself.

John Maynard Smith and Eörs Szathmáry called the great reorganisations of life **major transitions** in evolution: changes in how information is stored and passed on, in which things that used to copy themselves separately became parts of a larger whole (genes joining into chromosomes, cells into multicellular bodies). Every major transition raises the same question in a new form: how does a collection of things that copy become a single thing that copies itself?

2. Why we can't just look

Nobody can watch the origin of life. It happened about four billion years ago and left no fossils of its first steps. Chemists have built remarkable molecules, for example pairs of RNA enzymes that copy *each other* (Tracey Lincoln and Gerald Joyce, 2009). RNA is DNA's chemical cousin, and it matters because it can be two things at once: a message, like DNA, and a machine that does chemistry, like a protein. Many researchers think life began with RNA for exactly that reason: a molecule that is both instructions and machinery can, in principle, copy itself. Keep that double role in mind; our programs have it too. But no experiment has yet shown, step by step, how self-contained replicators first appear from a chemical mess that has none.

Two further ideas from origin-of-life research will matter later.

- **Tar.** Prebiotic chemistry (the chemistry before life) is messy. The same reactions that make useful molecules such as sugars also make a brown, sticky by-product that chemists call “asphalt” or “tar”, and it tends to win. The chemist Steven Benner has called this the asphalt problem: life had to start among its own debris.
- **The error threshold.** In 1971 Manfred Eigen showed that a copying system can maintain a message only up to a certain length before copying errors erase it. The more errors per letter, the shorter the message that survives. Life had to start short.

Part II. What was known: artificial life

3. Life in a computer

Artificial life (ALife) studies life-like processes in systems we build ourselves, often computers. The point is not to claim that a program is alive. It is to ask which processes *any* evolving system can or must go through, by building systems simple enough to watch completely, rerun, and take apart.

Classic ALife worlds such as **Tierra** (Tom Ray, 1991) and **Avida** (Charles Ofria, Chris Adami, Richard Lenski and colleagues; their 2003 Nature paper showed how complex functions evolve step by step) start from a replicator *designed by a human*. They tell us about evolution once life exists. They cannot tell us how the first replicator arose, because the experimenters put it there. Earlier experiments with randomly assembled code, such as Steen Rasmussen's Coreworld (1990), John Koza's random programs in the Lisp language, and Walter Fontana's “algorithmic chemistry” of interacting mathematical functions, showed that organised, self-maintaining patterns can appear from noise.

In 2024 a team led by **Blaise Agüera y Arcas** at Google showed something striking. You do not need to design the first replicator. Fill a computer's memory with *random* programs, let pairs of them run together again and again, and self-copying programs appear on their own and take over. They called it “computational life”. Their main world used a tiny programming language (a variant of the joke language Brainfuck, called BFF, which has only a handful of one-character commands), and they reported similar results with the instruction sets of real chips. (An **instruction set** is the full vocabulary of commands a processor understands.) One of those chips was the **Z80**. There they saw programs that copy by writing through the chip's scratchpad memory (the “stack”, explained below), followed later by programs that copy with a single “copy-this-whole-block”

instruction.

In 2026 another Google-led team (**Cicala** and colleagues) measured that takeover in Z80 soups: the first replicators, which work by “load a value, then write it out” (“load–push”, explained in Part V), are replaced by block copiers. This happens even when no program is rewarded for doing anything useful. They explained the replacement by **robustness**: the idea that the newcomers keep working better after a random change to one of their bytes.

What nobody had asked was the question of Part I. Are the first replicators individuals? If not, what changes when they are replaced? Where do the replacements come from: are they descendants of the first replicators, or newcomers that push them out? And what decides any of this?

Part III. Our world

4. A primordial soup of Z80 machine code

The ingredients:

- **The Z80** is a real 8-bit microprocessor from 1976 (8-bit means it handles one byte at a time), the chip inside the ZX Spectrum home computer and many arcade machines. We do not use a physical chip: we use an exact software copy of it (an **emulator**) running on graphics cards, checked against an independent reference.
- **Machine code** is a program written directly as bytes. Some bytes are **instructions** (also called opcodes), such as “add”, “copy” or “jump”; others are **data** (operands) that an instruction uses. The same byte can be either: it is an instruction if the processor’s finger lands on it, and data if an earlier instruction grabs it as input. This is the RNA property from section 2: the same stuff is both message and machine.
- **An address** is a byte’s position number in memory, like a house number on a street.
- **Registers** are a few tiny storage slots inside the processor that hold the numbers it is working on. They have names such as BC and HL.
- **A tape** is a short string of bytes, typically 16 to 64 long: a tiny genome that is also a program.
- **The soup** is a grid of 160×125 squares. Each square, which we call a **cell** (a grid square, not a living cell), holds one tape: 20,000 tapes in all, filled at the start with random bytes. Each cell has four neighbours (up, down, left, right), so the soup has geography, like a petri dish.
- **An encounter.** Two neighbouring tapes are glued into one shared memory (the first tape, then its **partner**), and the emulated Z80 runs that memory as a program for 128 instructions, starting at the first byte of the first tape, with every register set to zero. Whatever the program writes stays written. Then the two tapes are put back.
- **A step** is one round of encounters: at every step, 8,192 randomly chosen cells each meet one neighbour. A **world** is one run of the soup from random noise, often for hundreds of thousands of steps.
- **Mutation.** At each step a few hundred random bytes somewhere in the soup are replaced by random values.
- **The instruction pointer** (or program counter) is the processor’s finger: the address of the next byte it

will run as an instruction. We can follow the finger exactly and record *where* a program executes.

- **The stack** is the processor’s scratchpad. A “push” writes two bytes onto it; a “pop” or a “return” reads two bytes back. At the start of each encounter it points at the very end of the shared memory, which is the end of the *partner’s* tape, and it fills backwards from there. This detail turns out to matter a great deal.

There is no **fitness function** (no score the experimenters use to reward programs), and no goal. Programs that happen to write copies of themselves into neighbours become more common, because the copies copy too. That is natural selection with nothing added.

5. How we know something is alive: the culture test

Counting common patterns is not enough, as we found the hard way (Part IV). So we measure heredity by **intervention**, the way a microbiologist cultures a sample. We lift a tape out of the soup, run it against fresh random partners, and then run its *copies* against fresh partners again. If the copies of its copies still resemble it, the tape is **heritable**. We also check that its “copies” really are copies, at least three quarters identical, because some patterns can spread by smearing fragments without copying anything. The share of randomly chosen cells that pass is the population’s **heritable fraction**. This test is our own yardstick; we checked that the open beginning described below holds whether the bar for “heritable” is set lower or higher.

Part IV. How we got here

6. A bug, a lost observation, and an atlas

Algocell began as a web application that runs this soup live on a graphics card (a **GPU**, a chip that does thousands of small calculations at once), built on our own emulator of the Z80, which we call Zilion.

The research question started from an observation. Some weeks earlier, in the app, we had switched off the Z80’s copying instructions and seemed to see a strange new kind of life emerge. When we went back to it on 7 October, we found that the switch had never actually blocked the Z80’s block-copy instruction, LDIR (one instruction that copies a whole run of bytes, repeating by itself until done). LDIR is spelled with two bytes, and our switch only ever checked the first. Once we fixed that, with every copying instruction truly removed, no replicator appeared in any of 40 worlds run for a million steps each. The observation we remembered, which had block copying still switched on, we have not been able to re-establish.

That setback became a method. We built an **ablation atlas** (ablation means removing a part to see what depends on it): remove one group of related instructions at a time, and see whether, when and how life still comes. Over the following days we ran, on rented graphics cards, more than 3,600 worlds, varying the instruction set, the tape length, the number of instructions per encounter, the mutation rate and the layout of memory. The decisive experiments were **pre-registered**: before running them, we wrote down our predictions and the results that would count as proving us wrong (**kill criteria**).

What the atlas showed:

- **Three parts are load-bearing.** Remove the instructions that load a number written right after them in

the program (**immediate loads**) and only 2 of 10 worlds produce a replicator. Remove the instructions that write through the stack and the wait for life becomes 160 to 992 times longer, and the life that comes is a different kind, built on block copying. Remove all copying and nothing comes. Removing any other group of instructions changed the waiting time by a factor between 0.75 and 6.5, with no statistically convincing effect.

- **Order comes before life, and fools the detectors.** The first thing to form in a fresh soup is not alive. Some instructions leave bookmarks on the stack: a **call** jumps to another part of the program and leaves the address to come back to (a **return address**); a **reset** is a one-byte call to a fixed address. Run with every register empty, these instructions and pushes write mostly zeros, and zero is itself an instruction that does nothing. So zeros spread by being written and change nothing when run. When the most common pattern first holds a tenth of the soup, 24–34% of all bytes are zero, at every tape length from 9 to 100. We call this flood **tar**, after the chemists’ asphalt.

The tar fools the measures people have used to spot the origin of life in such soups. One looks for the moment the soup becomes repetitive, measured by how well it compresses, as a file compresses into a zip archive (its **high-order entropy**). It works on 16-byte tapes but is no better than chance, or even backwards, at six of eight longer lengths, because tar is repetitive too. Another comes from **assembly theory** (Lee Cronin, Sara Walker and colleagues), which looks for objects that take many steps to build and yet are found in many copies, a proposed signature of selection. It fires before the first replicator in 71 of 80 worlds. And the first replicator is as simple as the tar: it needs the fewest possible building steps for its length. Only intervention, testing what a sample *does*, tells life from debris.

- **Less can be more.** On 9-byte tapes, removing the stack-writing instructions made life *more* likely: 19 of 20 worlds against 5 of 20 (with 512 instructions per encounter; pre-registered and confirmed). The culprits are the call and reset instructions, whose return addresses spread as sterile smears, patterns that copy no heredity, and wreck programs about five times as often. The smears occupy the places where a block copier would have to be put together. Tar does not so much kill established life as stop new life from being assembled.
- **Heredity without clones.** On 100-byte tapes with only 64 instructions per encounter, no single encounter can copy a whole tape. Yet in 6 of 10 worlds, 69–100% of random cells passed the culture test while no single exact tape (no **genotype**) held more than 0.07% of the soup, about 14 cells out of 20,000. Each encounter copied a piece, and pieces were reassembled across the soup: the population, not any one tape, carried the heredity. Remember this in Part VI.
- **Eigen’s error threshold, drawn by programs.** Many replicators are a short pattern repeated along the tape (more on this in Part V). In the block-copy worlds, raising the mutation rate about a hundredfold shrank the repeating pattern the replicators could maintain from a median of 32.5 bytes to 5: the error threshold, emerging on its own.

Along the way, looking closely at the first replicator, we noticed what it does with the processor’s finger. That observation became the paper.

Part V. What we found

7. The first replicators are not individuals

When a soup first comes alive, the winner is almost always a two-byte **word** (our term for a short byte sequence) repeated end to end along the tape, like floor tiles: a **tiling**, whose **period** is the length of the repeating word. The commonest word is 01 c5, and there are close variants such as 21 e5. On tapes of 16 to 64 bytes with harmless tar, a word of this kind came first in 170 of 170 worlds, typically within 300–750 steps. There are 65,536 possible two-byte words (256×256). Exactly five of them, besides the trivial all-zero word, copy themselves perfectly into an empty partner, and all five are of this load-and-push kind. It is nearly the only thing of its size that replicates.

Here is how it works. 01 means “load the next two bytes into register pair BC”; the next two bytes are c5 01, part of the tape itself. c5 means “push BC”, which writes those two bytes where the stack points: at the end of the partner. Then the next 01 loads again and the next c5 pushes again, each time two bytes further back along the partner. The program is its own data. It copies itself as a side effect of being run. It has no loop, no counter and no test of its surroundings.

We call these first replicators **pushers**, and we call them **open**, because when we follow the instruction pointer it runs off the end of the pusher’s own tape and into the partner’s: in every one of 256 recorded encounters, for every one of the 100 first replicators we traced in worlds with harmless tar. A pusher uses its neighbour as its writing surface *and* then keeps executing whatever it finds there. That makes it sloppy.

- Against random partners it makes a good copy in only about two thirds of encounters (median 0.66–0.73). In a median 6–43% of encounters it loses a quarter or more of its own bytes, because after writing its copy it runs into the partner’s random bytes, which can spoil its register before the next push.
- Its children vary with the neighbour. On 16-byte tapes the child is an exact copy in 58% of encounters; on 50-byte tapes, in 5%. We can measure how much of the child is decided by the partner rather than the parent, in bits: 3.9 bits on 16-byte tapes, rising to 8 bits on 64-byte tapes, the most our measurement (with 256 partners) can register.
- Only 9–19% of random cells in the population pass the culture test, for thousands of steps.
- Its family lines do not last when tested alone. A **lineage** is a family line: a tape, its copies, their copies, and so on. In a **serial transfer** we copy a tape into a fresh random partner, then copy that copy into another fresh partner, and so on, like passing a culture from dish to dish. Copied this way, 96–100% of pusher lineages stop copying within eight transfers.

How do pushers persist at all, then? In the soup (on 16-byte tapes), more than half of a pusher’s encounters (57%) are with its own copies, since its neighbours are mostly its own offspring, and there it copies almost perfectly (98%). With strangers it is damaged in about a third of encounters (34%). So pushers persist as a crowd of near-copies, constantly damaged and constantly re-copied, rather than as clean family lines.

So the first replicator has heredity, but it is not an individual. What its child looks like depends on its neighbour, and its lines survive mainly in the company of its own kind. It is reminiscent of Godfrey-Smith’s scaffolded reproducers, with one difference: unlike a virus, the pusher brings its own copying code. What

it depends on is the neighbour's memory as a writing surface, and the neighbour's bytes, which it ends up running, shape how the copy turns out.

8. Then the loop closes

The first replicator does not stay. On 16-byte tapes, by 300,000 steps, the most common tape in 20 of 20 worlds carries a **control-flow instruction** (an instruction that decides where the finger goes next, such as a jump, a loop or a return, rather than computing or copying). In 17 of these worlds it is the very same eight-byte word, `ad e3 21 e3 21 c0 ad c0`, repeated twice.

Its key instruction is a **return**. A return sends the finger to an address stored on the stack (normally, it is how a program comes back from a call). This one is *conditional* (`c0`, which programmers write as RET NZ): it acts only if a check passes. The address it reads is made of the two bytes the program has just written onto the stack itself, and those bytes point back into the program's own body. The program's own product becomes its own control: the finger is sent home, and it never leaves. (Recall the RNA idea: the same bytes are message and machine.)

We call this **closure**, and the replicators that achieve it **closed**, **self-confined**, or **closers**. Three things travel together here, and we measured them separately:

- **a loop**: the code sends the finger back to where it has been (or uses a block-copy instruction, which loops by itself);
- **confinement**: the finger never runs a partner byte as an instruction (89 of 89 loop-bearing final winners);
- **independence**: the children no longer depend on the partner. In 63 of 67 loop-bearing final winners the child is one and the same string in all 256 encounters, and the heritable fraction of random cells rises to 75–94%. Confinement and independence agree in 106 of 110 final winners.

Why does the closer win? Not by robustness: 74% of a pusher's **single mutants** (versions with exactly one byte changed) are still heritable, against 59% of the closer's, on 16-byte tapes. What the closer gains is independence. It copies into every one of 256 random partners without damaging itself, and in serial transfer none of its unmutated family lines is lost, against 96–100% for the pusher. Its children are copies of it whatever neighbour it meets.

Whenever a closer arises, it spreads. Planted in 1% of the cells of a world full of pushers (**seeded**, in our jargon), the evolved closer held half the cells within 400–750 steps in 5 of 5 worlds; a pusher seeded into a closer's world vanished in 5 of 5. Longer tapes close more slowly: 12 of 20 worlds on 32-byte tapes within a million steps, 8 of 20 on 64-byte tapes. We also built an “8080-like” machine, keeping only the instructions of the Z80's predecessor, the Intel 8080, and turning the rest into “do nothing” instructions, to check that our results are not a quirk of one chip. On it, no world closed on 32-byte tapes in a million steps, even though a closer exists there and takes over when we plant it. **The barrier is discovery, not possibility or competition.** Evolution has to find the closer.

Why must a self-confined replicator contain a loop? A counting argument: the **pigeonhole principle** (if you put more pigeons than holes, some hole gets two). An encounter in our main experiments runs 128 instructions, and our tapes have at most 100 bytes. If every instruction comes from inside your own tape, and

nothing stops the program early, then some address must be visited twice: the finger has come back to where it was. Staying home forces a return. (The argument proves only that the finger revisits a spot, not that the whole program repeats exactly; and under lethal tar, where a program *can* stop early, it does not apply.)

The counting also tells us what it *cannot* explain. One might think open replicators come first simply because there are more of them among all possible programs. They are not more numerous. Of all programs built from a repeating four-byte word that rewrite themselves perfectly into an empty partner, 558 of 641 are closed block copiers. Open replicators come first because evolution finds them faster: a median of 450 steps to the first replicator against 21,000 to the first closed one, on 16-byte tapes. That head start is something we measured; no formula of ours predicts it.

In one sentence: heredity comes before individuality. The first things that evolve copy themselves by spilling into their neighbours. Self-contained copiers come later.

9. The rules of the world decide how life begins

Is “open first” a law, or a property of this one machine? We changed the machine and the environment. (We sometimes call the world’s basic rules its **substrate**, or loosely its “chemistry”: which instructions exist, and what its debris does.)

The literal channel. The pusher works because the Z80 has a cheap way to carry a number inside a program and stamp it out into memory: an immediate load followed by a push. A number written into the program itself is called a **literal**, so we call this the **literal channel**. It is how a program can carry its own text and print it elsewhere. BFF, the Brainfuck-like language of the 2024 study, has none. BFF reads and writes memory through two movable pointers called **heads**, and its commands only add one to a byte, subtract one, move a head, or copy a byte from one head to the other. Copying a 64-byte tape into a partner that differs from it everywhere therefore takes at least 128 commands: 64 that write a byte and 64 that move a head. A program that stays inside its own 64 bytes can only do that by running some of its commands more than once. That is a loop, by the same pigeonhole argument. And indeed, in our BFF soups every first replicator contained a loop (28 of 28). Then we added a single command that writes out its own two following bytes: a literal channel. In 12 of 12 worlds the first replicator became that command repeated end to end, loop-free and open, its finger entering the partner in every encounter. It is the 01 c5 of BFF. **One instruction decides whether life begins open.**

The tar’s toxicity. Then the open BFF replicator died. BFF marks loops with brackets, and a bracket without its partner halts the program, so BFF’s debris is *lethal*: it stops any program that runs into it. Open replicators run into everything, and within about a thousand **epochs** (BFF’s version of a step, in which every program meets one other) they held under 1% of the soup. Make the brackets harmless, and the open replicator held the soup for all 16,384 epochs in 12 of 12 worlds. The open phase is a **window** whose length the tar sets.

The same holds for the Z80. On the real Z80 a zero byte is harmless. We built a variant in which running a zero stops the encounter (**lethal tar**), and a **lethality dial** in which a zero stops it with probability p : $p = 0$ means never, $p = 1$ means always, $p = 0.3$ means 30% of the time. With lethal tar, no pusher established itself in 10 of 10 pre-registered worlds, and life began closed and late, as a block copier, after a median of 38,000 steps against 450 with harmless tar. On the dial, the first replicator is open in 10 of 10 worlds at every p up to 0.1, in 2 of 10 at $p = 0.3$, and in none at $p = 1$.

A simple model in the paper ties these together. Think of every open replicator alive at a given moment as a lottery ticket for finding a closer: the more open replicators there are, and the longer they last, the more likely a closer has been found. Harmless tar keeps the window open until a closer is found; lethal tar shuts it first. Two honest caveats: the dial supports this picture only at its two ends (closure timing in the middle does not follow it neatly), and the model was written with point mutations in mind, whereas our newest result (section 13) shows that the first closers are written by programs that cannot copy themselves. The tickets, it seems, are chances for such a program to write a closer from words the open replicators keep in circulation.

In one sentence: the rules of the world decide the order. A world with a cheap literal channel and harmless debris passes through a communal, open phase. A harsh world starts with self-contained replicators, or with nothing.

10. Inside the individual: one byte decides between self-repair and evolution

Once replicators are closed, a new question arises: what do they pass on?

We took 14 replicators, made every possible single mutant of each, and followed each mutant through eight serial transfers. In the closers that took over our worlds, 60–88% of mutant lineages lived on, and in every one of them the mutation had been *erased*. These closers copy the code they run and rebuild the rest of the tape from it. A different design, a short copying **core** (the few bytes that do the copying) followed by bytes it copies but never runs, kept the mutation in 46–63% of mutant lineages.

The difference turns on a recurring four-byte core, `XX 5e ed b0`. It is one instruction (`XX`), then “load a byte” (`5e`), then `LDIR (ed b0)`, the block copy. Here is the trick. Because every register starts at zero, the “load a byte” instruction reads the tape’s first byte, `XX`, into a register, and the block copy uses that register to decide how far ahead to write. So the value of the first byte sets where each copy lands. We call it the **copy offset**, d : each byte is copied d positions ahead of itself. Because the copy runs forward over memory it has just written, it keeps re-copying its own first d bytes, like a rubber stamp walking down a page. One byte becomes a switch.

- If d is small (say 4), the replicator stamps its four-byte core over the whole tape, its own and the partner’s. Anything else that was there, including any mutation, is wiped. These are **regenerators**: they rebuild themselves every generation, and so they cannot pass on variation.
- If d equals the tape length, the replicator copies its whole tape as it is, including $d - 4$ bytes it never runs (the 4 is the core; the other $d - 4$ bytes are passengers, copied but never run). A mutation in a passenger is inherited. These are **transmitters**.

We proved (it is “Proposition 8” in the paper; a proposition is a proved mathematical statement) that whenever d divides the tape length evenly, a lineage can pass on mutations at exactly $d - 4$ positions. The data agree: of 2,343 block-copy cells sampled from our worlds, scanning every single mutant finds exactly $d - 4$ positions that pass on in 2,289 (98%). The stamping trick is a known programming idiom, and Cicala and colleagues had labelled that first byte the “partner offset”. What is new is that evolution uses it as a switch between repair and heredity, and that we can count its consequences.

Why this is deep. Variation is the raw material of evolution. A regenerator stays alive but cannot change:

every mutation is repaired away. Only a transmitter keeps a store of changes that selection can later use. Biology has a famous distinction between the material that is inherited (the **germline**) and the body that is rebuilt each generation (the **soma**); a transmitter's passengers are a minimal germline. In the 1940s the mathematician John von Neumann worked out what a self-reproducing machine needs: a description of itself that it uses twice, once followed as instructions and once copied blindly as data. Transmitters have the blind-copy half: text that is copied without being read. So the first step to individuality does not simply improve heredity. It can change its kind: the closers that hold soups with harmless tar are mostly regenerators, which have traded variation for repair.

11. The environment chooses, through the company you keep

Which kind wins? It depends on the tar. With harmless tar, regenerators are the majority of heritable cells in 17 of 20 worlds on 16-byte tapes, at the end of the run. With lethal tar, transmitters are the majority in 9 of 10 worlds, carrying a median of 11.5 passenger positions each, while the most common tape holds only 0.2–1.5% of the soup. **Life that begins closed under lethal tar begins with passengers:** a swarm of slightly different versions rather than copies of one tape. On 32-byte tapes, whether we start a soup with 1%, 50% or 99% transmitters, it settles at 3–37% transmitters with harmless tar and 55–87% with lethal tar.

Our first two pre-registered explanations of this were wrong (section 12). So this week we took 20 settled soups, ran each for 2,000 steps under each rule (40 runs), recorded every encounter, and did the bookkeeping: who copied whom, who was destroyed, who was converted. This is an after-the-fact (post hoc) description, not a tested prediction, but it is revealing.

- Regenerators and transmitters copy *each other* in balanced numbers (to within 0.1%). Head to head, neither has an edge.
- The differences lie in their encounters with the **broken background**: the cells with no working core (a median 13% of the soup), the soup's debris.
 - A regenerator's body is its core repeated eight times. To a broken program whose finger wanders into it, this is a **trap**: the intruder starts running the copy loop with its own garbage settings, and in most such encounters (75–89%, depending on the soup) the regenerator is destroyed. Occasionally the trap works the other way and converts the intruder into a regenerator (in 0.5–0.9% of encounters).
 - A transmitter's body is mostly passengers. To an intruder it is a **maze**: the intruder wanders, and with lethal tar it is often stopped by a zero before it does harm. So lethal tar protects transmitters more than regenerators.
- Added up, these terms account for the shift we see when a soup is moved from one rule to the other: transmitters gain under the lethal rule and lose under the harmless one. This is an accounting check on the same runs, not an independent prediction.

What this does *not* yet explain is why regenerators hold the majority under harmless tar in the first place; the paper lists the full mechanism of the environment's choice as an open question. The direction of the story is clear, though: the environment does not reward a strategy directly. It changes how each kind of body fares against the debris around it.

12. A beautiful idea that turned out to be wrong

Science is also the story of hypotheses that die. Here is one we liked very much.

Transmitters copy bytes they never run. Under lethal tar, a zero hidden among those passengers would be harmless to its owner, which never runs it, but deadly to any intruder that wanders in. Could transmitters be carrying **poison in their junk**, a defence that works only on others? It would have been a lovely example of what Richard Dawkins called an **extended phenotype**: a trait whose effect shows up outside the body that carries it.

We pre-registered predictions and tested them. The idea failed its main tests.

- The zeros are not where intruders enter. They pile up towards the *end* of the tape (up to 32–38% at the second-last byte on 32-byte tapes), exactly where pushed garbage lands. They are **scars** from past intrusions, which transmitters inherit and regenerators erase.
- They are as common with harmless tar as with lethal tar (6.7% against 5.2% on 32-byte tapes), and absent without mutation.
- Removing them changes a host's survival by about 0.001 per encounter: real, but twenty times below the threshold we had set in advance.

Transmitters carry the record of the damage done to them. That is interesting, but it is not a weapon.

Two other guesses died the same night. *Backup cores*: perhaps a regenerator survives damage because its spare copies of the core take over. They don't: a regenerator whose first core is damaged never copies itself (0 of 4,096 cases). *Zeros as a shield*: perhaps the inherited zeros are what protects transmitters under lethal tar. They aren't: removing them keeps 88–94% of the protection. A fourth test, of whether each step along a gradual path from pusher to closer is favoured, was uninformative, because the soup is a swarm of mutants rather than a set of clean types. All of these are recorded in our pre-registration log, and they are being written into the revised paper.

13. The newest finding: how the first individual is born

The question every reviewer of our paper asked was the most basic one: **do the self-confined replicators descend from the open ones, or do they arise independently and push them out?** And if they descend, how: by a series of small mutations, or some other way? Earlier work could not say. Cicala and colleagues tracked only which group a program belonged to, not who its parents were.

Building a perfect record. We built a recorder that writes down *every* encounter in a world as it happens: which two tapes met, what each looked like before and after, and which bytes changed by mutation. It had to be live, because the GPU simulation is not exactly repeatable: two runs from the same starting point drift apart, because thousands of pairs are drawn at the same time and the order in which they claim cells varies. From the record we can rebuild the **line of descent** of any tape: its parent, its parent's parent, and so on, back to the random soup at step 0. One subtlety: because an encounter can splice bytes from two tapes, a new tape can have two parents. We validated the recorder before trusting it. Replaying 795,934 recorded encounters reproduced every byte, with zero mismatches; and in a test world where we planted a closer among pushers,

all 64 sampled lines of descent traced back to the planted closer.

Getting the analysis right took several rounds, and it is worth saying so plainly, because this is how science actually works. The pre-registered test turned out to be uninformative (see the base-rate point below), and the analysis that gives the result was refined after we had looked at the data. So treat this result as exploratory: strong, but not yet confirmed by a fresh, pre-registered run.

- Our first rule for “this tape copied that one” counted mere damage as copying in soups where neighbours already looked alike. We caught it by testing the supposed founders directly: they could not copy at all. We amended the rule (a copy must make at least a quarter of the bytes newly match the copier) and wrote the amendment down before rerunning.
- Our first rule for “parent” followed whichever of the two parents supplied more bytes. When a pusher cell is gradually overwritten by a closer, that rule follows the pusher’s material, not the closer’s function. We switched to the **functional parent**: the parent that supplied the bytes the new program actually runs.
- An independent reviewer of our draft figure found a bug in where each story “begins”, and pointed out that we had mixed the *first* founder in each world with later look-alikes made by existing closers. We fixed both. We also withdrew a claim (“the founders’ bytes were mostly written by tapes that cannot copy themselves”) because such tapes are most of the soup, so most bytes come from them by chance.

What the record shows. We recorded 20 worlds of 16-byte tapes from random soup onwards; 17 became self-confined. In each, we found the **founder**: going back along the line of descent of the closed population, the earliest ancestor that already stays home, counted from the most recent ancestor that was still an open copier.

- **Every founder’s line runs back to open pushers.** This was almost bound to be so: by then, essentially every living cell’s ancestry passes through an open copier within the previous 2,000 steps. So descent is observed, but on its own it is weak evidence; the informative part is *how* the founder was made.
- **The first founder was never a single-byte change to a working copier.** In none of the 17 worlds was it a **point mutation** (a random change of one byte) of a tape that could already copy itself. It appeared a median of 79 steps after its most recent open-copier ancestor (between 1 and 360 steps).
- **It was written by a program that could not copy itself.** We replayed the exact encounter that made each founder. In all 17 worlds, the program that ran in that encounter was a **non-copier**: a tape that fails the copying test. It *wrote* the founder, into its partner in 13 worlds and into its own half in 4; a median of 7 of the founder’s 16 bytes were in neither tape before, so this is writing, not splicing. There were two routes. In 11 worlds an open non-copier, its finger running into its partner, wrote the closer in that one encounter; with other partners it almost never does (at most 19% of random partners). In 5 worlds a *confined* program that writes the closer into any partner, and turns itself into the closer, had appeared one to three steps earlier, made by a point mutation in two and a rewrite in three; it does not count as a copier only because it does not copy the half of itself it never runs. (In one more world, the block copier, an open program did the same.)
- **Its parts are the pusher’s own words, slightly changed.** The pusher 21 e5 means “load register HL; push HL”. Change one byte and you get 21 e3 (“load HL; *swap* HL with the top of the stack”) or 21 e0 (“load HL; *return*, if a check passes”). Swapping with the stack lets a program pick up the bytes it has

just written; returning sends the finger to them. Those are exactly the two ingredients of a closer. Before each founder appeared, the words it runs were carried by roughly eight times as many cells as random tapes would carry (a median of 0.21% of cells, against 0.024%), as debris of the pushers. A non-copier that pushes these words in the right order writes a return closer, such as 3d e3 21 e3 21 e0 3d e0 (a cousin of the closer in section 8: the same pattern with a few bytes changed). In 15 of the 17 worlds, the first founder carries this pattern; one was a block copier and one a push-and-return hybrid.

- **Later founders are different.** Once a closer exists, new self-confined copiers in the same world (15 of them) mostly arise when an existing closer overwrites part of a neighbour: 11 of the 15 took bytes from a closer. The first individual is written by non-individuals; later ones are mostly made by individuals.

We also asked, in an exploratory test, whether evolution *could* have taken the slow road. We took a pusher and an evolved return closer and built every in-between tape you can make by taking, at each position where they differ, either the pusher's byte or the closer's (4,096 to 65,536 tapes per pair; four pairs). Then we asked whether a chain of single-byte changes leads from the pusher to a heritable closer with every step still heritable. Heritable closers exist among these tapes (43 to 163 per pair), but in three of the four pairs no such chain reaches one; in the fourth, a six-step chain does. Within these sets, at least, the heritable closers sit on islands that single changes mostly cannot reach without passing through tapes that cannot copy.

So, in one sentence:

The first self-confined replicator in each of these worlds was not copied or mutated from a replicator. It was written, in one encounter, by a program that could not copy itself, from words the sloppy open replicators keep in circulation.

A note on honesty. Our first version of this result said the founder was “assembled by recombination”. An independent critic showed that the category we had called recombination simply meant “neither a copy nor mostly the old tape”, and that in several worlds the founder's predecessor already wrote it. Replaying the founding encounters gave the more precise answer above, and we withdrew the first wording.

Part VI. Why this matters

14. For how we think about the origin of life

1. **Heredity without individuals is not a paradox; in some worlds it is the default.** Our first replicators have heredity but their children depend on their neighbours, and their lines survive mainly among their own kind. Evolution works on them anyway. The “individual” is not a precondition for Darwinian evolution. It can be one of its products.
2. **We can watch something like Woese's Darwinian threshold, byte by byte.** Woese's picture of a communal era giving way to individual lineages was a theory about ancient cells. Here there is a communal phase: sloppy copiers whose children depend on their neighbours, and, in the extreme case of Part IV, populations that carry heredity with no clone at all. There is a pool of shared words kept in circulation

by those copiers. And there is the moment when a program that cannot copy itself writes, from that pool, the first copier whose children depend only on itself.

3. **The first individual can be made by non-individuals.** We usually picture the first self-copier as a lucky mutant of an earlier copier. Here it was written by programs that could not copy themselves, out of material the sloppy copiers had spread around. The communal soup, with all its sideways traffic, is not just noise that individuality must overcome; it is where the first individual is made.
4. **The path depends on the chemistry.** A cheap literal channel and harmless by-products give a communal, open beginning. Toxic by-products, or no literal channel, give a beginning with loops and self-contained replicators. Origin-of-life researchers argue about whether life began with genes, with cells, or with **metabolism** (self-sustaining cycles of chemical reactions). Some of that argument may really be about which environment one has in mind.
5. **Individuality is a measurable quantity, and a loop comes before a wall.** In these worlds the first “self” is not a membrane. The organism’s bytes are as exposed as ever. It is a loop of control: a path of execution that returns into the organism, so that what it produces depends on itself and not on its neighbour. We can measure it as the information that flows from the neighbour into the child, which falls to zero when the loop closes. In BFF we even found independence without confinement: a replicator built from a literal command that writes two or three copies of its word at once enters its partner every time, yet its children carry nothing of the partner. The essence of individuality is that your future is independent of your surroundings. A boundary is one way to get that, not the only way.
6. **Order is not life, and detecting life at its origin needs intervention.** The tar flood is orderly, compresses well, and fools complexity and assembly measures. The first replicator is as simple as the tar. Only lifting a sample out and testing what it does tells them apart. That is a warning for anyone searching for the beginnings of life, on Earth, in a laboratory or on another world, with measures of order alone.

15. For evolution in general

7. **The first step to individuality can cost variation.** One byte separates repair from **evolvability** (the capacity to vary and so to evolve further), and under harmless tar the closers that win are mostly regenerators that erase their mutations. The regenerator-versus-transmitter split is a minimal model of a germline: some inherited material is copied without being used, and only the lineages that keep it can pass on change. Which strategy wins is decided by the environment, not by any foresight about the future.
8. **Bodies matter even for the smallest replicators.** Whether a body is a trap or a maze for intruders, and whether the surroundings are toxic, changes how each kind fares, even when the copying machinery is identical.
9. **Debris shapes what is possible.** By-products that spread without heredity can prevent life from assembling (the 9-byte result in Part IV), set how long an open phase lasts, and help decide between kinds of heredity.

16. For artificial life

10. **Origin questions can now be answered experimentally.** Random machine code, an exact recorder and a few hours of rented computing give a complete fossil record of an origin of individuality: every ancestor, every encounter, every byte. No chemical experiment can do that yet.
11. **Test heredity by intervention, and classify randomly chosen tapes, not the most common pattern.** Several of our own early conclusions were overturned when we did. A regenerating population is nearly a clone, so its most common tape is representative. A transmitting population is a swarm in which no tape is common, so its most common tape misleads.

17. Predictions for chemists

These are predictions for chemistry, not results about it.

- **Template copying is a literal channel.** In DNA and RNA each chemical letter attracts its partner letter (A with T or U, G with C), so a strand prints its mirror image directly, with no machinery to read it, just as a pusher prints its own bytes. So the first replicators of such a chemistry should be short repeats that template themselves, depend on their surroundings, and make mostly partial copies.
- **By-products are a variable.** Whether a chemistry's tar is inert or actively blocks copying should decide whether an open phase lasts long enough for control to evolve. Matched experiments with inert and blocking additives could test it.
- **Where a copy restarts matters.** If a copying molecule restarts partway through the unit it is copying, it should regenerate a short repeat (a regenerator); if it restarts at the unit's end, it transmits the whole unit (a transmitter).
- **Made before self-made.** If the analogy holds, the first self-sustaining replicators should first appear as products of molecules that do not themselves replicate, not as point mutants of earlier replicators.

Part VII. Honesty and limits

18. What we are careful not to claim

- **This is a computer, not chemistry.** The Z80 was designed by engineers, and its instruction set shapes what is easy and what is hard. We tested the Z80, an 8080-like subset of it, and one toy language (BFF).
- **Our "individuality" is one measurable piece** of what biologists mean: whether a copier's activity, and the information in its children, stay its own. Real organisms also maintain themselves, regulate their boundaries and much more.
- **The simulation's built-in rules give replicators a leg up.** Tapes of fixed size, a fixed starting point, and every register set to zero at the start of each encounter are imposed by the simulation (its "harness"), and the evolved closers rely on the empty registers. With random starting registers, closure still came, but more than a hundred times more slowly.
- **Closure is not universal within the time we ran.** On 50-byte tapes, the closed tape is only a passing

stage: its offspring re-open. On 64-byte tapes, most worlds had not closed after a million steps.

- **The newest result is new and exploratory.** It rests on 17 worlds of one tape length with harmless tar, the analysis was refined after looking at the data, it is under independent review, and the figure that shows it is being redrawn after the critic's comments.
- **Credit where due.** The takeover of load–push replicators by block copiers in Z80 soups was reported by Agüera y Arcas and colleagues (2024) and measured by Cicala and colleagues (2026). Our contributions are what the takeover *is* (a transition from open to self-confined execution, measured by following the finger), what decides whether and when it happens, what it does to heredity, and how the first self-confined replicator is born.

19. How the work was done

The confirmatory experiments were pre-registered with kill criteria, and the misses are reported. The prediction that living among kin is what sustains the open phase was killed: in **well-mixed** soups, where partners are drawn from anywhere rather than from next door, the open phase is just as heritable, and geography only makes closure more reliable. Our prediction that a pusher's mutations persist failed. The poison, backup-core and shield ideas failed. Several measurement rules were found wanting on contact with the data and amended in writing before rerunning, and analyses done after looking at data are labelled as such. An independent critic reviews every figure before it is shown. Every number in the paper, and in this document, comes from a table generated by code in the project repository (the paper draft manuscript/MAIN_nature.md, the tables under results/, and the outcome sections of the pre-registration log REVISION_PREREG.md).

20. Where this goes next

- **Is the first individual always written by non-copiers?** The same exact recording under lethal tar, where life begins closed, and on 32-byte tapes, where the closers are block copiers; and a fresh, pre-registered run to confirm the founder result.
- **A faithful 8080.** The real Intel 8080 treats some bytes differently from our “8080-like” machine; a faithful copy would make the second-machine comparison cleaner.
- **Individuality as information, throughout.** Our records make it possible to measure, for every lineage and every moment, how much of a child is self and how much is neighbour, and to watch that quantity cross its threshold.

Glossary

8080-like: our reduced Z80 that keeps only the instructions of its predecessor, the Intel 8080, turning the rest into “do nothing” instructions.

Ablation: removing a part (here, a group of instructions) to see what depends on it.

Address: a byte’s position number in memory.

Artificial life (ALife): the study of life-like processes in systems we construct, often computers.

Assembly theory / assembly index: a theory (Cronin, Walker and colleagues) that measures how many joining steps, at minimum, an object needs to be built from basic parts; many copies of an object with a high index is proposed as a sign of selection.

BFF: the Brainfuck-family language of Agüera y Arcas and colleagues (2024): a handful of one-character commands that move two heads, change bytes, copy between heads, and loop with brackets.

Bit: the unit of information; the answer to one yes-or-no question.

Block copier: a replicator that copies itself with LDIR, the Z80’s copy-a-block instruction (bytes ed b0).

Byte: a number from 0 to 255; the unit of our tapes. Written in hexadecimal as two characters (00 to ff).

Call / reset / return: a call jumps elsewhere and leaves a return address on the stack; a reset is a one-byte call to a fixed address; a return jumps to the address on the stack.

Cell: one square of the soup’s grid, holding one tape.

Closer, closed, self-confined: a replicator whose instruction pointer never leaves its own tape. We also measure separately whether its children are independent of its partner; the two agree in 106 of 110 cases. Philosophers’ “operational closure” is broader.

Communal evolution: Woese’s picture of early life as loosely organised cells exchanging genes so freely that there were no stable lineages.

Control flow: instructions that decide where the processor’s finger goes next (jumps, loops, calls, returns).

Copy offset (*d*): how far ahead a block copier’s copy lands; set by the first byte of its core.

Core: the few bytes of a replicator that do the copying.

Culture test: our heredity test: a tape is heritable if the copies of its copies still resemble it, against fresh random partners.

Darwinian threshold: Woese’s term for the transition from communal early life to life organised in individual lineages.

Emulator: software that imitates a chip exactly.

Encounter: one run of two neighbouring tapes, glued into one memory, for a fixed number of instructions.

Epoch: BFF’s version of a step, in which every program meets one other.

Error threshold: Eigen’s limit on how long a message a copying system can maintain at a given error rate.

Evolvability: the capacity of a lineage to vary, and so to evolve further.

Extended phenotype: Dawkins’s term for effects of genes outside the body that carries them.

Fitness function: a score the experimenters use to decide which programs reproduce; our soups have none.

Founder: the earliest ancestor on a line of descent that already stays home, counted from the most recent open-copier ancestor.

Functional parent: when a new tape has two parents, the parent that supplied the bytes the new tape actually runs.

Genotype: one exact sequence of bytes.

Germline and soma: the inherited material of an organism, and the body that is rebuilt each generation.

GPU: a graphics chip that does thousands of small calculations at once.

Harness: the fixed rules of the simulation (tape size, starting point, starting registers).

Heredity / heritable: offspring resemble their parents because information is copied; a tape that passes the culture test.

Heritable fraction: the share of randomly chosen cells that pass the culture test.

Hexadecimal: writing numbers in base 16 with the digits 0–9 and a–f; each byte is two characters.

High-order entropy: a measure of how predictable, or compressible, the soup’s contents are.

Horizontal transfer: genetic material passing between organisms other than from parent to child.

Immediate load: an instruction that loads the number written right after it in the program.

Individual (here): a replicator whose copying is self-contained and whose children depend on itself, not on its neighbour.

Instruction pointer (program counter): the processor’s “finger”, the address of the next byte it will run.

Instruction set: the full vocabulary of commands a processor understands.

Kill criterion: a result, written down before an experiment, that would count as refuting the hypothesis.

LDIR: the Z80 instruction that copies a block of memory byte by byte, repeating by itself until done.

Lethal tar / lethality dial: variants in which running a zero byte stops the encounter, always or with probability p .

Line of descent: the chain of ancestors of a tape, back to the start.

Lineage: a family line: a tape, its copies, their copies, and so on.

Literal / literal channel: a number written into the program itself; a cheap way for a program to stamp such numbers out into memory (in the Z80, an immediate load followed by a push).

Machine code: a program written as raw bytes that a processor runs directly.

Major transition: one of the great reorganisations in the history of life, in which the way information is stored and passed on changes and formerly separate replicators become parts of a larger whole (Maynard Smith and Szathmáry).

Median: the middle value of a set of numbers.

Metabolism: self-sustaining cycles of chemical reactions that build and power a living thing.

Mutation (point mutation): a random change of one byte.

Open replicator (pusher): a replicator whose activity runs into its partner’s tape; it copies itself by writing into its neighbour.

Opcode / operand: a byte read as an instruction, and a byte read as an instruction’s data.

Partner: the second tape in an encounter.

Period / tiling: a tiling is a tape made of one short word repeated end to end; the period is that word’s length.

Pigeonhole principle: if there are more pigeons than holes, some hole gets two; here, more executed instructions than bytes means some byte is run twice.

Post hoc: decided after looking at the data, and so weaker evidence than a pre-registered test.

Pre-registration: writing down predictions and kill criteria before seeing the data.

Protocell: a hypothetical early cell: a membrane bag holding molecules that copy themselves.

Non-copier: a tape that fails the copying test: it does not make copies of itself against random partners.

Rewrite: our name for a new tape that is neither a copy of its partner nor mostly its own old bytes.

Register: a tiny storage slot inside the processor (the Z80 has pairs such as BC and HL).

Regenerator: a self-confined replicator that copies only the code it runs and rebuilds the rest of its tape from it, erasing mutations (for example, the block copier with a small copy offset, and the evolved 16-byte return closer).

Replicator: anything that makes copies of itself that can in turn make copies.

Return closer: a closer whose loop is made by a return instruction that jumps back into its own body.

Robustness: the ability to keep working after a random change.

Scaffolded reproducer: something that reproduces only with outside help (Godfrey-Smith).

Seeded: deliberately planted in a soup by the experimenters.

Serial transfer: copying a tape into a fresh partner, then copying that copy, and so on.

Single mutant: a version of a tape with exactly one byte changed.

Stack: the processor's scratchpad, written by push and read by pop and return; in our soup it starts at the end of the partner's tape and fills backwards.

Step: one round of encounters in the soup (8,192 of them).

Substrate: the underlying medium of a living system (a chemistry, or here a machine and its rules).

Tape: a short string of bytes; one organism in the soup.

Tar: zero bytes and other debris that spread in the soup without heredity; named after prebiotic chemistry's asphalt.

Transmitter: a self-confined replicator that copies its whole tape, including passenger bytes it never runs, and so passes on mutations.

Well-mixed: a soup in which partners are drawn from anywhere, not just next door.

Word: our term for a short byte sequence, especially one that repeats along a tape.

World: one run of the soup from random noise.

Z80: an 8-bit microprocessor from 1976; our soup runs an exact software copy of it.

Further reading

Agüera y Arcas, B. et al. Computational life: how well-formed, self-replicating programs emerge from simple interaction. *arXiv:2406.19108* (2024).

Benner, S. A., Kim, H.-J. & Carrigan, M. A. Asphalt, water, and the prebiotic synthesis of ribose, ribonucleosides, and RNA. *Acc. Chem. Res.* 45, 2025–2034 (2012).

Cicala, F. et al. Coevolution of self-replication and function in a digital primordial soup. *arXiv:2607.09211* (2026).

Dawkins, R. *The Extended Phenotype* (Oxford University Press, 1982).

- Eigen, M. Selforganization of matter and the evolution of biological macromolecules. *Naturwissenschaften* 58, 465–523 (1971).
- Godfrey-Smith, P. *Darwinian Populations and Natural Selection* (Oxford University Press, 2009).
- Krakauer, D. et al. The information theory of individuality. *Theory Biosci.* 139, 209–223 (2020).
- Lenski, R. E., Ofria, C., Pennock, R. T. & Adami, C. The evolutionary origin of complex features. *Nature* 423, 139–144 (2003).
- Lincoln, T. A. & Joyce, G. F. Self-sustained replication of an RNA enzyme. *Science* 323, 1229–1232 (2009).
- Maturana, H. R. & Varela, F. J. *Autopoiesis and Cognition* (Reidel, 1980).
- Maynard Smith, J. & Szathmáry, E. *The Major Transitions in Evolution* (Freeman, 1995).
- Vetsigian, K., Woese, C. & Goldenfeld, N. Collective evolution and the genetic code. *PNAS* 103, 10696–10701 (2006).
- von Neumann, J. *Theory of Self-Reproducing Automata* (ed. Burks, A. W.; University of Illinois Press, 1966).
- Woese, C. R. On the evolution of cells. *PNAS* 99, 8742–8747 (2002).